

Passer d'un écran à l'autre

Un titre, une partie, une fin — et le bouton qui mène de l'un à l'autre.

C'est le squelette de n'importe quel jeu : un écran d'accueil, la partie, l'écran de fin. Tout tient en trois pièces, et il n'y en a pas d'autres.

- 1 **Une variable dit où l'on est.** Un octet, `scene`, qui vaut `TITRE`, `JEU` ou `FIN`. Le reste du programme ne fait que la lire.
- 2 **Un seul endroit change d'écran.** `allerA()` éteint, efface, redessine, rallume. Si chaque bouton faisait son propre ménage, l'un finirait par oublier d'effacer, et deux écrans se mélangeraient.
- 3 **On ne redessine qu'au changement.** Redessiner à chaque image coûterait 360 cases soixante fois par seconde, pour rien : l'écran ne bouge pas tant que la scène ne change pas.

1. Ce que ça donne

Les quatre images ci-dessous sont l'écran de la console émulée, photographié après chaque appui sur `A`. Le quatrième appui ramène au titre : la roue se referme.



2. Le programme, morceau par morceau

LES VALEURS DE LA SCÈNE, ET LA SCÈNE ELLE-MÊME

```
uint8_t TITRE = 0;
uint8_t JEU = 1;
uint8_t FIN = 2;
uint8_t scene = 0;
uint8_t a = 0;
uint8_t aAvant = 0;
uint8_t x = 0;
uint8_t y = 0;
```

Trois noms plutôt que trois nombres : `scene == JEU` se relit, `scene == 1` demande de se souvenir. `aAvant` retient l'état du bouton à l'image précédente — on verra au chapitre 4 pourquoi il est indispensable.

EFFACER — LES 20 × 18 CASES DE L'ÉCRAN

```
void effacer() {
    y = 0;
    while (y < 18) {
        x = 0;
        while (x < 20) {
            poser(x, y, 0);
            x = x + 1;
        }
        y = y + 1;
    }
}
```

La tuile 0 est vide. Sans cet effacement, « A POUR JOUER » resterait sous « A POUR FINIR », les deux textes mélangés à l'écran.

UN DESSIN PAR ÉCRAN

```
void dessinerTitre() {
    texte(6, 5, "MON JEU");
    texte(4, 10, "A POUR JOUER");
}

void dessinerJeu() {
    texte(5, 4, "C EST PARTI");
    texte(4, 10, "A POUR FINIR");
}

void dessinerFin() {
    texte(7, 5, "FINI");
    texte(3, 10, "A POUR REVENIR");
}
```

Chacune ne sait que dessiner. Aucune ne décide de rien, aucune n'efface : c'est ce qui permet de les appeler depuis un seul endroit.

ALLERA — LE SEUL ENDROIT QUI CHANGE D'ÉCRAN

```
void allerA() {
    ecran(0);
    effacer();
    if (scene == TITRE) {
        dessinerTitre();
    }
    if (scene == JEU) {
        dessinerJeu();
    }
    if (scene == FIN) {
        dessinerFin();
    }
    ecran(1);
}
```

L'ordre compte. `ecran(0)` éteint l'affichage, on fait le ménage à l'abri, `ecran(1)` rallume. Repeindre 360 cases prend bien plus de temps qu'une image n'en contient : écran allumé, on verrait le ménage se faire.

```

int main() {
    scene = TITRE;
    allerA();
    while (true) {
        image();
        a = bouton(A);
        if (a == 1) {
            if (aAvant == 0) {
                if (scene == TITRE) {
                    scene = JEU;
                } else {
                    if (scene == JEU) {
                        scene = FIN;
                    } else {
                        scene = TITRE;
                    }
                }
            }
            allerA();
        }
        aAvant = a;
    }

    return 0;
}

```

`image()` attend la fin de l'image en cours. En dehors du changement de scène, cette boucle ne fait rien d'autre que lire un bouton : c'est normal, l'écran est déjà peint.

3. Pourquoi `ecran(0)` avant d'effacer

Une Game Boy affiche soixante images par seconde. Entre deux images, elle laisse un court instant — le *VBlank* — pendant lequel la mémoire vidéo est libre. Poser 360 tuiles demande beaucoup plus que cet instant.

Le piège : si on efface écran allumé, le balayage passe pendant le ménage et affiche un mélange des deux écrans — un scintillement, ou une bande de l'ancien texte qui survit une fraction de seconde. Éteindre l'écran suspend le balayage : plus rien ne s'affiche pendant qu'on travaille, et tout apparaît d'un coup au rallumage.

4. Le détecteur de front

C'est le point qui casse le plus souvent, et il ne se voit pas dans le code :

```

a = bouton(A);
if (a == 1) {
    if (aAvant == 0) {    // ← le front : A vient tout juste d'être pressé
        ...
        allerA();
    }
}
aAvant = a;              // ← on retient l'état pour l'image suivante

```

`bouton(A)` ne dit pas « A vient d'être pressé » : il dit « A est enfoncé *maintenant* ». Un appui humain dure une dizaine d'images. Sans `aAvant`, la scène avancerait dix fois — on traverserait les trois écrans et on retomberait sur le titre avant d'avoir eu le temps de lire quoi que ce soit.

La comparaison `a == 1 && aAvant == 0` n'est vraie qu'à **une seule image** : celle où le bouton passe de relâché à enfoncé. Un appui, une transition, quelle que soit sa durée.

5. Ce que la console a répondu

Ces trois lignes ne sont pas une promesse : la cartouche a été chargée dans l'émulateur, le bouton pressé, et `scene` relue dans la mémoire de la console.

- ✓ au moment de l'appui, la scène avance d'un cran

`scene = 1`

- ✓ A maintenu trois secondes : plus rien ne bouge

`scene = 1`

- ✓ relâché puis repressé : la scène avance encore

`scene = 2`

6. Pour t'en servir

Ajouter un écran tient en trois gestes : un nom de plus dans les valeurs de `scene`, une fonction `dessinerX()`, et un `if` de plus dans `allerA()`. Rien d'autre du programme ne change.

`exemples/ecrans.cpp` — le programme entier, 84 lignes.

`exemples/decoupe/` — le même, réparti sur cinq fichiers, pour voir `#include` à l'œuvre.

`livrets/lecon-18-passer-d-un-ecran-a-l-autre.pdf` — la même idée écrite avec `enum` et `switch`.

Fiche produite par `node fiche-ecrans.mjs` — le programme y est compilé, exécuté et photographié à chaque impression. Elle ne peut pas vieillir sans qu'on le voie.